AI CAREER FOR WOMEN
Engineer Spoke

Smart Human - Computer Interaction using Hand Gestures

A Project Report
submitted in partial fulfillment of the requirements
of
AICW project.

by

Dodda Vishali, vishalidodda22@gmail.com

Neha Adepu, nehaadepudoc@gmail.com

Fariha Naaz, farihanaaz2004@gmail.com

Sumana Sri Aashily, sumanasriaashily@gmail.com

Under the Guidance of

Rapaka Manoj Kumar

SME

Edunet Foundation

ACKNOWLEDGEMENT

We would like to take this moment to thank all those who have helped and supported us, directly or indirectly, in the completion of this thesis work.

First and foremost, we would like to thank our supervisor, Rapaka Manoj Kumar, for his constant support, suggestions, and encouragement throughout the entire project. His suggestions and criticisms have helped us in improving our work and thinking in new and innovative ways. His faith in us has encouraged us to do our best and complete this project successfully. It has been an honor and a great learning experience to work under his supervision. His supervision has not only helped us in the successful completion of this project but has also helped us in developing into more responsible and confident individuals.

Smart Human – Computer Interaction Using Hand Gestures system that enables users to interact with a computer using hand gestures instead of a traditional physical mouse. The system leverages computer vision techniques to detect and track hand movements in real time through a webcam. Using image processing libraries and hand landmark detection algorithms, the system identifies finger positions and interprets specific gestures to perform mouse operations such as cursor movement, left click, right click, scrolling, and drag actions. The implementation ensures smooth cursor control by mapping hand coordinates to screen coordinates while applying filtering techniques to reduce noise and enhance stability. This touchless interaction approach improves accessibility, supports hands-free computing, and demonstrates the practical application of human-computer interaction using real-time vision-based gesture recognition. The project highlights the integration of Python, OpenCV, and hand tracking frameworks to build an efficient, low-cost, and user-friendly virtual mouse system.

TABLE OF CONTENTS

Abstract
List of Figures
List of Tables

Chapter 1. Introduction

1.1 Problem Statement

1.2 Motivation

1.3 Objectives

1.4. Scope of the Project

Chapter 2. Literature Survey

Chapter 3. Proposed Methodology

Chapter 4. Implementation and Results

Chapter 5. Discussion and Conclusion

References

LIST OF FIGURES

Fig no	Figure name	Page No.
Figure 1	Gesture indicating index finger (cursor movement)	**20**
Figure 2	Gesture indicating thumb + index touch (drag : mouse hold + move)	**20**
Figure 3	Gesture indicating all fingers closed (release drag)	**21**
Figure 4	Gesture indicating thumb and pinky open (Exit Program)	**21**

CHAPTER 1

Introduction

1.1 Problem Statement

Traditional human-computer interaction primarily depends on physical input devices such as a mouse and keyboard for tasks like navigation, selection, scrolling, and clicking. While these devices are widely used, they are not suitable in sterile environments such as hospitals, laboratories, and healthcare facilities, where frequent physical contact can increase the risk of contamination and infection transmission. In medical and public settings where hygiene and touchless interaction are critical, shared input devices can become carriers of pathogens, posing serious health risks. Additionally, during surgical procedures or laboratory experiments, professionals may require computer interaction without physically touching devices to maintain sterility. The lack of reliable, cost-effective, and hygienic touchless interaction systems limits safe digital access in such environments. Therefore, there is a need for a vision-based, gesture-controlled system that enables contact-free computer operation, reducing contamination risks while ensuring efficient and intuitive human-computer interaction.

1.2 Motivation

The rapid advancement of technology has increased the demand for safer, more hygienic, and touchless methods of human-computer interaction. Traditional input devices such as the mouse and keyboard require constant physical contact, making them less suitable for sterile environments like hospitals, laboratories, and healthcare facilities.

In medical and laboratory settings, maintaining hygiene is critical. Frequently touched devices such as keyboards and mice can act as carriers of harmful microorganisms, increasing the risk of contamination and infection transmission. Healthcare professionals, surgeons, and lab technicians often need to access digital systems without breaking sterility during procedures. Conventional hardware devices do not effectively support such touchless interaction requirements.

Moreover, in public environments where multiple users share systems, minimizing physical contact can significantly improve safety and hygiene standards. This highlights the need for a contact-free and reliable alternative to traditional input methods.

Therefore, the motivation behind this project is to develop a real-time, vision-based gesture-controlled virtual mouse system that enables users to operate computers through simple hand movements without physical contact. By utilizing computer vision and hand-tracking techniques, the system aims to provide a hygienic, efficient, and cost-effective solution for touchless human-computer interaction in healthcare and other sensitive environments.

1.3 Objective

The primary objective of this project is to design and implement a real-time gesture-controlled virtual mouse system that enables users to interact with a computer using hand movements captured through a webcam. The project aims to provide an alternative to traditional hardware-based input devices by utilizing computer vision techniques for accurate hand detection and gesture recognition.

Specifically, the main objectives include:

- To develop a hand-tracking system capable of detecting and tracking hand landmarks in real time.
- To map hand movements to screen coordinates for smooth and precise cursor control.
- To implement gesture recognition techniques to perform mouse actions such as left click, right click, scrolling, and drag operations.
- To enhance system stability and accuracy by applying filtering and smoothing techniques to reduce noise and unwanted cursor movement.
- To ensure the system operates efficiently with minimal latency, enabling near real-time performance using standard hardware.

Overall, the project seeks to demonstrate how computer vision–based gesture recognition can create an accessible, cost-effective, and touchless human-computer interaction system that improves usability and inclusivity compared to conventional input methods.

1.4 Scope of the Project

This project concentrates on the design and implementation of a computer vision–based gesture-controlled virtual mouse system for real-time human-computer interaction. It focuses on hand detection, gesture recognition, and cursor control using a webcam without relying on traditional physical input devices. The system utilizes existing hand-tracking frameworks and image processing techniques to interpret hand movements and convert them into mouse operations.

The study involves analyzing gesture recognition accuracy, cursor stability, responsiveness, and overall system performance under different lighting conditions and backgrounds. The project primarily emphasizes understanding the practical implementation of vision-based interaction systems and their advantages in providing accessible and touchless control mechanisms.

The scope of the project includes:

- Implementation of real-time hand tracking and landmark detection.
- Mapping of hand coordinates to screen coordinates for cursor movement.
- Recognition of predefined gestures for mouse actions such as clicking, scrolling, and dragging.
- Application of smoothing and filtering techniques to enhance stability and reduce noise.
- Evaluation of system responsiveness and usability in real-time scenarios.

The system operates using a standard webcam and pretrained hand detection models, enabling development without extensive hardware requirements. This approach allows the project to focus on application-level implementation, experimentation, and performance evaluation.

However, the project does not include the development of custom deep learning models from scratch, integration with voice-based interaction, embedded hardware deployment, or large-scale commercial optimization. Future enhancements may include improving gesture diversity, increasing robustness under complex backgrounds, optimizing latency, and extending compatibility across different operating systems and devices.

CHAPTER 2

Literature Survey

2.1 Review of Relevant Literature

Human-Computer Interaction (HCI) has evolved significantly over the years, moving from traditional input devices such as keyboards and mice to more natural and intuitive interaction techniques. Early research in gesture recognition focused on hardware-based systems such as data gloves and sensor-equipped devices, which, although accurate, were expensive and less user-friendly. These systems required additional hardware components, limiting their scalability and accessibility for everyday users.

With the advancement of computer vision and machine learning, researchers began exploring vision-based hand gesture recognition using webcams and image processing algorithms. Techniques involving skin color detection, contour extraction, background subtraction, and fingertip tracking were widely studied to detect and interpret hand movements. However, these approaches were often sensitive to lighting conditions, complex backgrounds, and variations in skin tones, affecting their robustness and accuracy.

Recent developments introduced deep learning–based hand tracking frameworks and real-time landmark detection models that significantly improved gesture recognition accuracy and performance. Frameworks such as MediaPipe and other convolutional neural network (CNN)–based approaches enabled precise detection of hand landmarks, allowing smooth cursor control and reliable gesture classification. These methods reduced dependency on handcrafted features and enhanced adaptability across different environments.

Based on the existing research, gesture-controlled systems have proven to be effective for touchless interaction, gaming applications, virtual reality environments, and assistive technologies. However, challenges such as latency, stability, background noise, and computational efficiency still require attention. The present project builds upon these advancements by implementing a real-time, webcam-based virtual mouse system that focuses on accuracy, responsiveness, and accessibility without requiring additional hardware, thereby contributing to practical and cost-effective human-computer interaction solutions.

2.2 Existing Models, Techniques, and Methodologies

Several gesture recognition and virtual interaction approaches have been developed:

- **Data Glove–Based Systems:** Use sensor-equipped gloves for accurate tracking but require expensive hardware.
- **Color-Based Hand Detection:** Relies on skin color segmentation; simple but sensitive to lighting and background changes.
- **Contour and Fingertip Detection:** Uses edge detection and convex hull methods; lightweight but less reliable under occlusion.
- **Machine Learning–Based Methods:** Apply classifiers like KNN or SVM for gesture recognition; require manual feature extraction.
- **Deep Learning–Based Hand Tracking:** CNN and landmark-based models provide accurate, real-time hand detection without extra hardware.
- **Vision-Based Virtual Mouse Systems:** Map hand landmarks to cursor movement and mouse actions; require optimization for smooth performance.

2.3 Gaps and Limitations in Existing Solutions

Despite significant progress in gesture recognition research, several limitations remain:

- Hardware-based systems increase cost and reduce portability due to dependency on external devices such as data gloves or sensors.
- Traditional color and contour-based methods are highly sensitive to lighting variations, background complexity, and skin tone differences.
- Some machine learning approaches require extensive training data and manual feature extraction, limiting scalability and adaptability.
- Real-time systems may experience latency, jitter, or unstable cursor movement due to noise in hand tracking.
- Many existing solutions focus on limited gesture sets and lack smooth integration with full mouse functionalities such as drag-and-drop or continuous scrolling.

CHAPTER 3

Proposed Methodology

3. System Design

3.1 System Design Overview

The system is designed to perform real-time hand gesture recognition and convert detected gestures into mouse control operations using a webcam. It processes live video frames, detects and tracks hand landmarks, interprets predefined gestures, and maps them to corresponding cursor actions. The architecture focuses on accuracy, smooth cursor movement, and low-latency performance to ensure natural human-computer interaction.

The system operates through a sequence of modules including video capture, hand tracking, gesture recognition, cursor mapping, and mouse control execution. Each module contributes to transforming raw visual input into meaningful control commands.

HAND TRACKING FRAMEWORK

A hand tracking framework is a computer vision–based model that detects hands in images or video streams and identifies key landmark points such as fingertips and joints.

It is widely used in:
• Gesture recognition systems
• Virtual mouse and keyboard applications
• Augmented Reality (AR) and Virtual Reality (VR)
• Sign language recognition

Core Principle: Landmark-Based Detection

The framework works by detecting a hand region and predicting multiple 2D landmark coordinates. These landmarks represent specific points on the hand (e.g., fingertips, knuckles, wrist). By analyzing the relative positions of these points, gestures can be identified accurately.

Components include:

• **Video Input Capture:** Continuously captures frames from the webcam.

• **Hand Detection Module:** Identifies the presence and location of the hand within each frame.

• **Landmark Extraction:** Detects key points on the hand for precise movement tracking.

• **Gesture Recognition Logic:** Interprets finger positions and distances to identify predefined gestures.

• **Coordinate Mapping:** Converts fingertip positions into screen coordinates for cursor control.

• **Smoothing Mechanism:** Reduces noise and stabilizes cursor movement.

• **Mouse Control Module:** Executes system-level mouse operations such as click, scroll, and drag.

3.1.1 Hand Detection and Initialization

This module acts as the entry point of the system and is responsible for detecting the user's hand in the video stream.

Process

• The webcam captures live video input.
• Each frame is processed and converted into RGB format.
• The hand detection model identifies the hand region.
• Landmark coordinates are extracted for further processing.

Purpose

• Establish hand presence.
• Initialize tracking.
• Provide positional data for gesture analysis.

3.1.2 Gesture Recognition and Cursor Control

This module interprets hand movements and converts them into mouse actions.

Process

• Fingertip coordinates (primarily index finger) are tracked.
• Distance between specific fingers (e.g., index and thumb) is calculated.
• Predefined conditions determine gestures such as: cursor movement, right click, left click, copy and paste, scroll, tab switching, volume up/down etc.
• Cursor position is mapped to screen resolution.
• Smoothing filters reduce jitter.
• Mouse commands are executed using system control libraries.

Purpose

• Enable natural cursor movement.
• Provide touchless click and scroll functionality.
• Ensure stable and responsive interaction.

By combining real-time hand tracking with efficient gesture mapping, the system delivers a cost-effective and accessible alternative to traditional mouse devices.

3.2 Modules Used

The proposed Gesture-Controlled Virtual Mouse system is composed of multiple functional modules. Each module performs a specific task, ensuring smooth and accurate gesture-based interaction.

3.2.1 Video Capture Module

This module is responsible for acquiring real-time video input from the webcam.

Key Functions:
• Captures continuous video frames.

• Converts frames into suitable color format for processing.
• Maintains real-time frame rate for smooth interaction.

Outcome:
Provides live visual input for further hand detection and gesture analysis.

3.2.2 Hand Detection and Landmark Extraction Module

This module detects the hand and identifies key landmark points such as fingertips and joints. It forms the core of the gesture recognition system.

Key Functions:
• Detects hand region in each frame.
• Extracts multiple hand landmark coordinates.
• Tracks hand movement continuously.
• Handles minor variations in orientation and position.

Outcome:
Generates precise hand landmark data used for gesture interpretation.

3.2.3 Gesture Recognition Module

This module interprets specific hand gestures based on landmark positions and distances between fingers.

Key Functions:
• Identifies raised or folded fingers.
• Calculates distance between fingertips.
• Detects predefined gestures (move, click, scroll, drag).
• Maps gestures to corresponding mouse commands.

Outcome:
Converts hand movements into meaningful control actions.

3.2.4 Cursor Mapping and Smoothing Module

This module ensures accurate and stable cursor control.

Key Functions:

• Maps fingertip coordinates to screen resolution.

• Applies interpolation for smooth cursor movement.

• Reduces noise and jitter using filtering techniques.

• Improves precision and stability.

Outcome:

Provides smooth, real-time cursor motion aligned with hand movement.

3.2.5 Mouse Control Execution Module

This module sends control commands to the operating system to perform mouse actions.

Key Functions:

• Executes left and right clicks.

• Performs scrolling and drag operations.

• Interfaces with system-level mouse control libraries.

• Ensures low-latency response.

Outcome:

Enables full virtual mouse functionality without physical hardware.

3.3 Data Flow Diagram (DFD)

The Data Flow Diagram (DFD) illustrates how data moves through the Gesture-Controlled Virtual Mouse system and how each module contributes to generating the final mouse control output.

3.3.1 DFD Level 0 – Overall System

Input: Live Video Stream (Webcam)

Process: Gesture-Controlled Virtual Mouse System

Output: Cursor Movement and Mouse Actions

Flow:

Webcam Input → Gesture Processing System → Mouse Control Output

Description:

At the highest level, the system captures live video input, processes hand gestures, and produces corresponding mouse operations such as movement, click, scroll, and drag. The entire system is represented as a single process.

3.3.2 DFD Level 1 – Hand Detection Module

Flow:

Webcam → Frame Capture → Hand Detection → Landmark Extraction

Description:

• The webcam captures video frames.
• Frames are processed and converted to the required format.
• The hand detection model identifies the hand region.
• Landmark coordinates (fingertips, joints) are extracted.

Output: Hand landmark coordinates.

3.3.3 DFD Level 1 – Gesture Recognition Module

Flow:

Landmark Data → Gesture Logic → Action Identification

Description:

• Landmark positions are analyzed.
• Finger states and distances are calculated.
• Predefined gesture conditions are checked.
• Corresponding mouse action is determined.

Output: Identified mouse action (move, click, scroll, drag).

3.3.4 DFD Level 1 – Cursor Mapping and Execution Module

Flow:

Gesture Action → Coordinate Mapping → Smoothing → OS Mouse Command

Description:

• Fingertip coordinates are mapped to screen resolution.

• Smoothing filters reduce jitter.

• Final cursor position is updated.

• Mouse control commands are sent to the operating system.

Output: Real-time cursor movement and mouse operations.

3.4 Advantages

- **Touchless Interaction:** Enables hands-free computer control without requiring a physical mouse, promoting hygiene and convenience.
- **Improved Accessibility:** Assists individuals with limited mobility by providing an alternative method of interaction.
- **Cost-Effective Solution:** Operates using a standard webcam without the need for additional hardware such as data gloves or sensors.
- **Real-Time Performance:** Processes video frames efficiently to ensure smooth and responsive cursor movement.
- **User-Friendly Interface:** Uses simple and intuitive hand gestures for performing common mouse operations.
- **Reduced Hardware Dependency:** Eliminates reliance on physical peripherals, reducing wear and maintenance.

3.5 Requirement Specification

3.5.1 Hardware Requirements

- Processor: Intel i5 / AMD Ryzen 5 or higher
- RAM: Minimum 4 GB (8 GB recommended)
- Webcam: Integrated or external webcam for real-time video capture
- Storage: At least 2 GB free space

3.5.2 Software Requirements

- Operating System: Windows / Linux / Ubuntu
- Programming Language: Python 3.8 or higher
- Libraries:
    - OpenCV (for image processing)
    - MediaPipe (for hand tracking and landmark detection)
    - NumPy (for numerical operations)
    - PyAutoGUI (for mouse control operations)
- Development Tools:
    - VS Code / PyCharm / Jupyter Notebook

CHAPTER 4

Implementation and Result

Fig 1: Gesture indicating index finger (cursor movement)

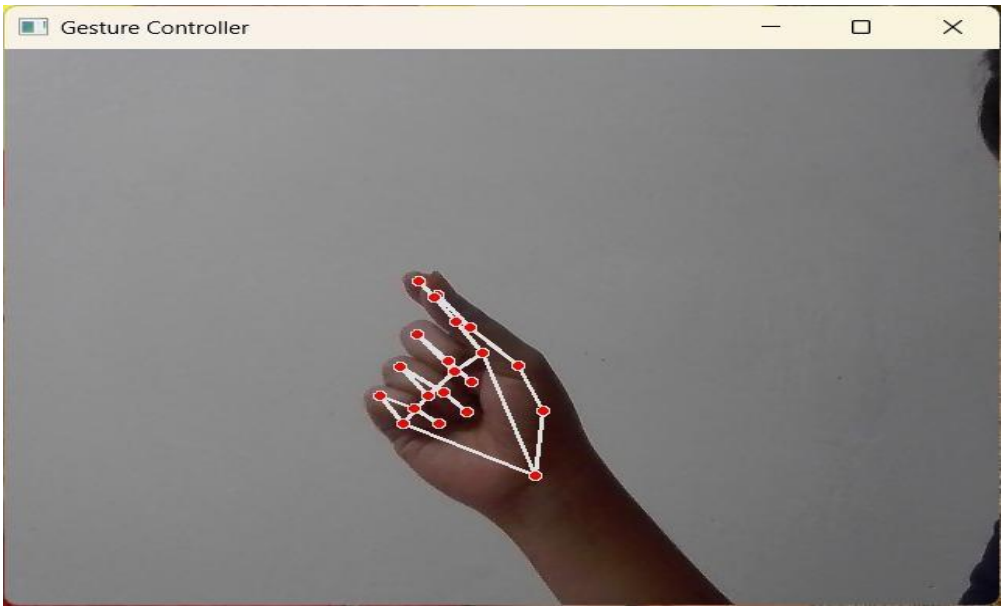

Fig 2: Gesture indicating thumb + index touch (drag : mouse hold + move)

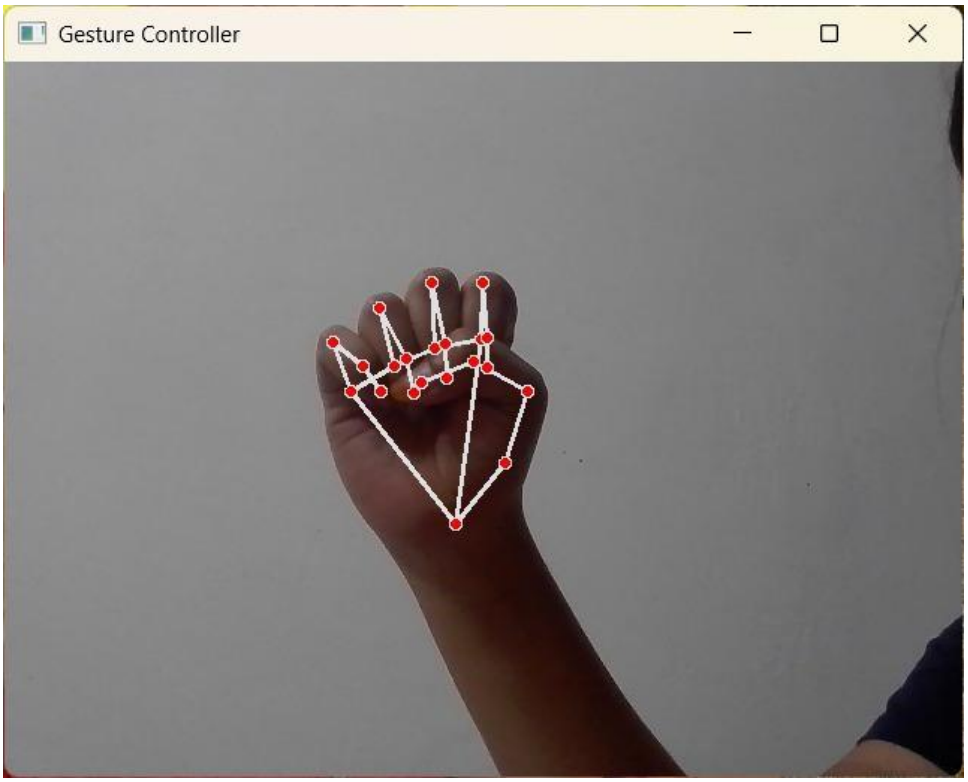

Fig 3: Gesture indicating all fingers closed (release drag)

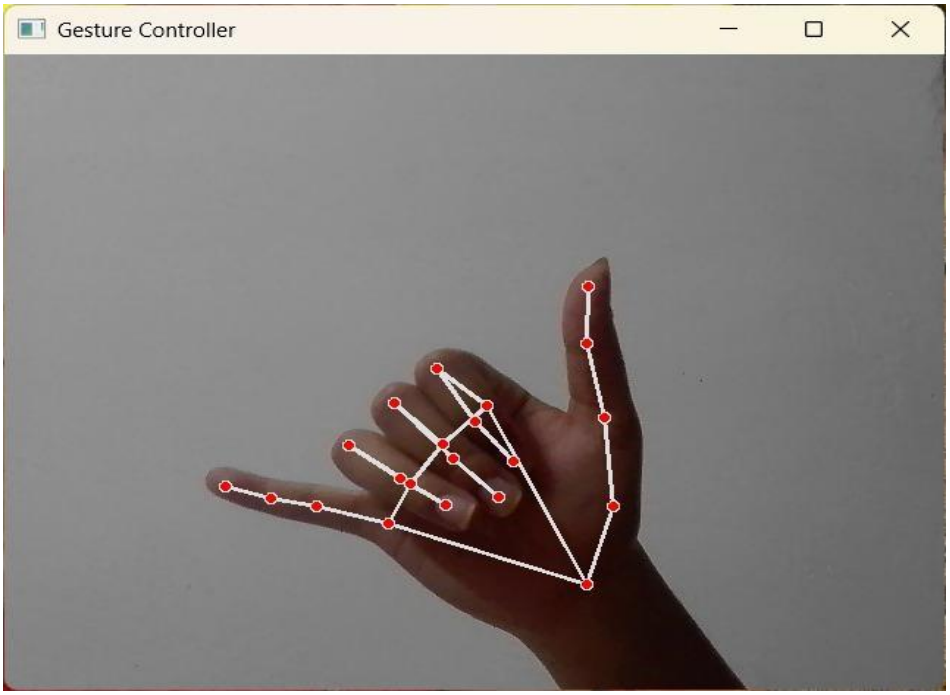

Fig 4: Gesture indicating thumb and pinky open (Exit Program)

CHAPTER 5

Discussion and Conclusion

5.1 Key Findings

- The system successfully enables touchless human-computer interaction using real-time hand gesture recognition.
- The MediaPipe hand tracking model accurately detects 21 hand landmarks, ensuring reliable gesture identification.
- The gesture recognition logic effectively interprets finger positions and distances to perform mouse and system control actions.
- The system supports multiple gestures for cursor movement, clicking, scrolling, dragging, volume, brightness, and tab control.
- The stability filtering mechanism reduces false gesture detection and improves accuracy.
- Real-time performance is achieved with smooth and responsive cursor control.
- The system works without physical devices, improving accessibility and hygiene.
- The integration of OpenCV, MediaPipe, and PyAutoGUI provides an efficient and scalable gesture-based control framework.

5.2 GitHub Link of the Project

https://github.com/Vishali-Dodda/Smart-Human-Computer-Interaction-Using-Hand-Gestures

5.3 Video Recording of Project Demonstration

The demonstration video link:

https://drive.google.com/file/d/1ecqWKjLEHvlOjApTlpL3knfTU-lhVszG/view?usp=drivesdk

Project Demo Video Link:

https://drive.google.com/file/d/18QFpnaCiNRkSPEPYZZfBpveDjbX3L4xe/view?usp=drivesdk

5.4 Limitations

- The system requires a webcam and proper lighting conditions for accurate hand detection.
- Performance may be reduced in low-light environments or complex backgrounds.
- Fast hand movements may lead to incorrect gesture detection.
- The system currently supports only single-hand gesture recognition.
- High-resolution processing may slightly affect real-time performance on low-end systems

5.5 Future Work

- Add multi-hand gesture support for more complex controls.
- Improve system performance under low-light and dynamic background conditions.
- Extend the system to mobile devices and IoT-based applications.
- Integrate with AR/VR systems for immersive interaction.

5.6 Conclusion

The gesture-based virtual mouse system provides an effective touchless human-computer interaction method using real-time hand gesture recognition. By utilizing MediaPipe hand tracking and computer vision techniques, the system accurately interprets finger movements to perform mouse and system control actions.

The integration of real-time gesture detection with system control functions demonstrates the potential of vision-based interaction systems as an alternative to traditional input devices. The project highlights the importance of natural and contactless interaction methods in modern computing environments.

Overall, the proposed system offers an efficient, user-friendly, and scalable solution for gesture-based control, making it suitable for smart environments, presentations, healthcare systems, and future touch-free computing applications.

REFERENCES

[1] https://developers.google.com/mediapipe/solutions/vision/hand_landmarker

[2] https://pyautogui.readthedocs.io/en/latest/

Appendices

Appendix A – Example Code Snippet

gesture_controller.py

```
import cv2
import mediapipe as mp
import pyautogui
import math
import time
from enum import Enum
from action_controller import ActionController
pyautogui.FAILSAFE = False
mp_drawing = mp.solutions.drawing_utils
mp_hands = mp.solutions.hands
# ---------------- GESTURES ---------------- #
class Gest(Enum):
    NONE = 0
    INDEX = 1
    PINCH = 2
    FIST = 3
    TWO = 4
    THREE = 5
    FOUR = 6
    CALL = 7   # Thumb + Little finger
# ---------------- HAND RECOGNITION ---------------- #
class HandRecog:
    def __init__(self):
        self.prev = Gest.NONE
        self.count = 0
        self.stable = Gest.NONE
    def get(self, hand):
        lm = hand.landmark
```

```
index = lm[8].y < lm[6].y
middle = lm[12].y < lm[10].y
ring = lm[16].y < lm[14].y
pinky = lm[20].y < lm[18].y
thumb_open = abs(lm[4].x - lm[2].x) > 0.15
palm_size = math.hypot(lm[0].x - lm[9].x,
                       lm[0].y - lm[9].y)
pinch_dist = math.hypot(lm[8].x - lm[4].x,
                        lm[8].y - lm[4].y)

pinch = pinch_dist < palm_size * 0.25
detected = Gest.NONE
# Exit Gesture (Thumb + Pinky only)
if thumb_open and pinky and not index and not middle and not ring:
    detected = Gest.CALL
elif pinch:
    detected = Gest.PINCH
elif index and not middle and not ring and not pinky:
    detected = Gest.INDEX
elif index and middle and not ring and not pinky:
    detected = Gest.TWO
elif index and middle and ring and not pinky:
    detected = Gest.THREE
elif index and middle and ring and pinky:
    detected = Gest.FOUR
elif not index and not middle and not ring and not pinky:
    detected = Gest.FIST
# Stability filter
if detected == self.prev:
    self.count += 1
else:
    self.count = 0
self.prev = detected
```

```
 if self.count > 5:
            self.stable = detected

        return self.stable
# ---------------- MAIN CONTROLLER ---------------- #
class GestureController:
    def __init__(self):
        self.cap = cv2.VideoCapture(0)
        self.hand = HandRecog()
        self.controller = ActionController()
        self.prev_x = None
        self.prev_y = None
        self.hold_start = None
        self.hold_gesture = None
    def start(self):
        with mp_hands.Hands(
                max_num_hands=1,
                min_detection_confidence=0.85,
                min_tracking_confidence=0.85) as hands:
            while self.cap.isOpened():
                success, frame = self.cap.read()
                if not success:
                    break
                frame = cv2.flip(frame, 1)
                rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
                results = hands.process(rgb)
                if results.multi_hand_landmarks:
                    hand = results.multi_hand_landmarks[0]
                    gesture = self.hand.get(hand)
                    sw, sh = pyautogui.size()
                    x = int(hand.landmark[9].x * sw)
                    y = int(hand.landmark[9].y * sh)
```

```
cur_x = hand.landmark[9].x
cur_y = hand.landmark[9].y
if self.prev_x is None:
    self.prev_x = cur_x
    self.prev_y = cur_y
delta_x = cur_x - self.prev_x
delta_y = cur_y - self.prev_y
# -------- EXIT (CALL Gesture Hold 2 sec) -------- #
if gesture == Gest.CALL:
    if self.hold_gesture != Gest.CALL:
        self.hold_start = time.time()
        self.hold_gesture = Gest.CALL
    elif time.time() - self.hold_start > 2:
        break
# -------- MOVE -------- #
elif gesture == Gest.INDEX:
    self.controller.move_mouse(x, y)
# -------- DRAG -------- #
elif gesture == Gest.PINCH:
    self.controller.drag_mouse(x, y)
# -------- RELEASE -------- #
elif gesture == Gest.FIST:
    self.controller.release_mouse()
# -------- TWO (Tabs + Copy) -------- #
elif gesture == Gest.TWO:
    if delta_x > 0.12:
        self.controller.next_tab()
    elif delta_x < -0.12:
        self.controller.prev_tab()
    if self.hold_gesture != Gest.TWO:
        self.hold_start = time.time()
        self.hold_gesture = Gest.TWO
    elif time.time() - self.hold_start > 2:
        self.controller.copy()
```

```
                self.hold_gesture = None
                # -------- THREE (Click + Paste) -------- #
                elif gesture == Gest.THREE:
                    if self.hold_gesture != Gest.THREE:
                        self.hold_start = time.time()
                        self.hold_gesture = Gest.THREE
                        self.controller.left_click()
                    elif time.time() - self.hold_start > 2:
                        self.controller.paste()
                        self.hold_gesture = None
                # -------- FOUR (Scroll) -------- #
                elif gesture == Gest.FOUR:
                    if abs(delta_y) > 0.03:
                        self.controller.scroll(delta_y)
                else:
                    self.hold_gesture = None
                self.prev_x = cur_x
                self.prev_y = cur_y
                mp_drawing.draw_landmarks(
                    frame, hand, mp_hands.HAND_CONNECTIONS)
            cv2.imshow("Gesture Controller", frame)
            # ESC backup exit
            if cv2.waitKey(1) & 0xFF == 27:
                break
        self.cap.release()
        cv2.destroyAllWindows()
if __name__ == "__main__":
    GestureController().start()
```

action_controller.py

```
import pyautogui
import time
pyautogui.FAILSAFE = False
class ActionController:
    def __init__(self):
        self.drag = False
        self.last_action = 0
        self.cooldown = 0.3
    def ready(self):
        now = time.time()
        if now - self.last_action < self.cooldown:
            return False
        self.last_action = now
        return True
    # -------- Mouse -------- #
    def move_mouse(self, x, y):
        pyautogui.moveTo(x, y, duration=0.05)
    def drag_mouse(self, x, y):
        if not self.drag:
            pyautogui.mouseDown()
            self.drag = True
        pyautogui.moveTo(x, y, duration=0.05)
    def release_mouse(self):
        if self.drag:
            pyautogui.mouseUp()
            self.drag = False
    def left_click(self):
        if self.ready():
            pyautogui.click()
    # -------- Scroll -------- #
    def scroll(self, delta):
        if delta > 0:
            pyautogui.scroll(-200)
```

```
    else:
        pyautogui.scroll(200)
    # -------- Copy / Paste -------- #
    def copy(self):
        if self.ready():
            pyautogui.hotkey("ctrl", "c")
    def paste(self):
        if self.ready():
            pyautogui.hotkey("ctrl", "v")
    # -------- Tabs -------- #
    def next_tab(self):
        if self.ready():
            pyautogui.hotkey("ctrl", "tab")
    def prev_tab(self):
        if self.ready():
            pyautogui.hotkey("ctrl", "shift", "tab")
```

This code demonstrates video loading, prompt input, and mask generation.

Appendix B – System Environment

- **Programming Language:** Python
- **Frameworks/Libraries:** OpenCV, MediaPipe, PyAutoGUI
- **Model Used:** MediaPipe Hand Tracking (Pre-trained Hand Landmark Model)
- **Hardware:** Webcam-enabled system with standard CPU (GPU not mandatory)

Appendix C – Additional Observations

- Gesture detection accuracy improves with proper lighting conditions.
- Stable hand positioning increases gesture recognition reliability.
- System performance depends on webcam quality and system processing speed.
- Background complexity may affect hand detection accuracy.